

Vietnam National University, Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science & Engineering



BÁO CÁO BÀI TẬP MỞ RỘNG

MÔN: HỌC MÁY (CO3117)

**ĐỀ TÀI: XÂY DỰNG MÔ HÌNH BAYESIAN NETWORK
VÀ CÁC THUẬT TOÁN SUY LUẬN CHÍNH XÁC VÀ SẮP XỈ**

LỚP: TN01

GVHD: Lê Thành Sách

Student	Student ID
Huỳnh Đức Nhân	2312420
Nguyễn Thiện Minh	2312097
Nguyễn Lưu Khánh Trình	2313638
Lê Công Vinh	2313912



Bảng phân công công việc

STT	Mã số SV	Họ và tên	Nhiệm vụ được phân công	Kết quả	Ký tên
1	2312420	Huỳnh Đức Nhân	Viết code cho BTL1 và BTL2	100%	
2	2312097	Nguyễn Thiện Minh	Viết code cho mở rộng, viết mô tả code cho BTL1	100%	
3	2313638	Nguyễn Lưu Khánh Trình	Viết code cho BTL3, viết mô tả cho BTL1 và một phần BTL3	100%	
4	2313912	Lê Công Vinh	Viết code cho BTL3 và mở rộng, viết mô tả code cho BTL3 và báo cáo	100%	



Mục lục

1	Giới thiệu bài toán	3
2	Xây dựng thư viện Bayesian NetWork	3
2.1	Xây dựng lớp Graph	3
2.2	Xây dựng lớp BayesianNetwork	5
3	Tổng quan về dữ liệu	10
3.1	Nguồn dữ liệu	10
4	Tiền xử lý số liệu	12
4.1	Kiểm tra Missing value	12
4.2	Kiểm tra số lượng giá trị khác nhau	13
5	Thống kê tả đối với các biến numeric	14
5.1	Đánh giá tổng quan	14
5.2	Phân tích biểu đồ boxplot	16
6	Thống kê tả đối với các biến phân loại	18
7	Lựa chọn các cột cho mô hình	19
8	Huấn luyện mô hình	22
9	Kết luận	25
10	Tài liệu tham khảo	26

1 Giới thiệu bài toán

Trong đề tài này, nhóm tiến hành xây dựng và hiện thực mô hình Mạng Bayes (Bayesian Network) nhằm thực hiện suy luận xác suất trên dữ liệu. Cụ thể, nhóm triển khai hai phương pháp suy luận chính gồm suy luận chính xác (Exact Inference) và suy luận xấp xỉ cơ bản (Approximate Inference). Mục tiêu ứng dụng của mô hình là dự đoán khả năng một người có mắc chứng trầm cảm hay không, dựa trên các mối quan hệ nhân quả giữa các yếu tố ảnh hưởng trong dữ liệu.

Để thực hiện, nhóm sử dụng bộ dữ liệu Student Depression Dataset từ Kaggle, bao gồm 27.901 mẫu với 18 đặc trưng (feature) ở cả hai dạng số (numeric) và phân loại (categorical). Dữ liệu chứa các thông tin liên quan đến suy nghĩ tự tử, áp lực học tập, căng thẳng trong công việc, mức độ hài lòng trong học tập, cùng nhiều yếu tố khác có liên quan đến sức khỏe tâm lý của sinh viên. Thông qua mô hình này, nhóm hướng đến việc khám phá quan hệ nhân quả giữa các yếu tố, đồng thời đưa ra suy luận hợp lý về nguy cơ trầm cảm của cá nhân dựa trên bằng chứng quan sát được.

2 Xây dựng thư viện Bayesian NetWork

2.1 Xây dựng lớp Graph

Trong phần này, nhóm tiến hành xây dựng một lớp Graph bằng ngôn ngữ Python nhằm mô phỏng và thao tác với đồ thị có hướng (Directed Graph). Lớp này được thiết kế với mục tiêu cung cấp các chức năng cơ bản nhưng quan trọng trong việc biểu diễn và xử lý cấu trúc đồ thị, đặc biệt phục vụ cho các bài toán về **phụ thuộc nhân quả** hoặc **mạng Bayes (Bayesian Network)**.

Lớp Graph hỗ trợ các chức năng chính sau:

- **Thêm node và cạnh có hướng:** thông qua các phương thức `add_node()` và `add_edge()`, cho phép xây dựng cấu trúc đồ thị động một cách linh hoạt.
- **Truy xuất mối quan hệ cha – con:** bằng hai hàm `get_parents()` và `get_children()`, hỗ trợ việc xác định cấu trúc phụ thuộc giữa các biến trong đồ thị.
- **Sắp xếp topo (Topological Sort):** được hiện thực trong hàm `topological_sort()`, giúp sắp xếp các node trong đồ thị có hướng không chu trình (DAG) theo thứ tự phụ thuộc hợp lý.
- **In cấu trúc đồ thị:** thông qua phương thức `__str__()`, hiển thị đồ thị dưới dạng danh sách kề, giúp dễ dàng quan sát và kiểm tra quan hệ giữa các node.

Nhờ các chức năng trên, lớp Graph đóng vai trò là mô-đun nền tảng, có thể mở rộng và ứng dụng trong nhiều bài toán khác nhau như:

- Mô hình **Mạng Bayes (Bayesian Network)** cho phân tích xác suất và suy luận nhân quả.
- Bài toán **lập lịch (scheduling)** và quản lý phụ thuộc giữa các tác vụ.
- Các **thuật toán duyệt đồ thị** hoặc **phân tích cấu trúc** trong mạng phụ thuộc.

```
1 class Graph:
2     def __init__(self):
3         # adjacency list: {node: [list of children]}
4         self.adj = {}
5         self.back = {}
6
7     def add_node(self, node):
8         """Them node moi vao graph"""
9         if node not in self.adj:
10             self.adj[node] = []
11             self.back[node] = []
12
13     def add_edge(self, u, v):
14         """Them canh co huong u -> v (u la parent, v la child)"""
15         if u not in self.adj:
16             self.add_node(u)
17         if v not in self.adj:
18             self.add_node(v)
19         self.adj[u].append(v)
20         self.back[v].append(u)
21
22     def get_parents(self, node):
23         """Tra ve danh sach cha (parents) cua node"""
24         return self.back.get(node, [])
25
26     def get_children(self, node):
27         """Tra ve danh sach con (children) cua node"""
28         return self.adj.get(node, [])
29
30     def topological_sort(self):
31         """Sap xep topo cac node trong DAG"""
32         visited = set()
33         stack = []
34
35         def dfs(u):
36             if u in visited:
37                 return
38             visited.add(u)
39             for v in self.adj.get(u, []):
```

```
40         dfs(v)
41         stack.append(u)
42
43     for node in self.adj:
44         dfs(node)
45
46     stack.reverse()
47     return stack
48
49 def __str__(self):
50     return "\n".join([f"{u} -> {self.adj[u]}" for u in self.adj])
```

2.2 Xây dựng lớp BayesianNetwork

Trong phần này, nhóm hiện thực một lớp BayesianNetwork trong Python, kế thừa từ lớp Graph, nhằm mô phỏng **Mạng Bayes (Bayesian Network)** – một mô hình đồ thị xác suất biểu diễn mối quan hệ nhân quả giữa các biến ngẫu nhiên. Mô hình này cho phép suy luận và dự đoán dựa trên xác suất có điều kiện, đóng vai trò quan trọng trong các hệ thống trí tuệ nhân tạo và phân tích dữ liệu.

Lớp BayesianNetwork được thiết kế với các chức năng chính sau:

- **Thêm biến và bảng xác suất có điều kiện (CPT)** thông qua hàm `add_variable()`, cho phép định nghĩa cấu trúc mạng cùng các mối quan hệ xác suất giữa các biến.
- **Tính xác suất có điều kiện và xác suất chung** bằng các phương thức `probability()` và `joint_probability()`, giúp đánh giá xác suất của một tập hợp biến theo cấu trúc mạng.
- **Truy vấn suy luận xác suất**: phương thức `query()` thực hiện **suy luận chính xác (Exact Inference)** để tính phân phối hậu nghiệm $P(Q|evidence)$, trong khi `approximate_query()` sử dụng **phương pháp Monte Carlo Sampling** để ước lượng phân phối xấp xỉ.
- **Hỗ trợ tính toán log-xác suất** qua hàm `log_query()`, giúp tránh sai số tràn số khi làm việc với các xác suất rất nhỏ.
- **Trực quan hóa mạng Bayes** bằng hàm `visualize()`, cho phép hiển thị cấu trúc đồ thị cùng bảng CPT của từng node, hỗ trợ quá trình phân tích và trình bày mô hình.

Với các chức năng trên, lớp BayesianNetwork không chỉ là công cụ mô phỏng mà còn là nền tảng cho việc:

- Phân tích mối quan hệ nhân quả giữa các biến trong dữ liệu.
- Dự đoán trạng thái của một biến khi biết các bằng chứng liên quan.

- Ứng dụng trong các hệ thống như **chẩn đoán y tế**, **dự đoán hành vi**, **hệ chuyên gia**, và nhiều bài toán xác suất khác.

Như vậy, việc xây dựng lớp BayesianNetwork tạo nên nền tảng quan trọng để nhóm có thể triển khai các thuật toán suy luận xác suất trong bài toán dự đoán trầm cảm của sinh viên dựa trên dữ liệu thực tế.

```
1 class BayesianNetwork(Graph):
2     def __init__(self):
3         super().__init__()
4         self.cpt = {} # {node: {(parent_values): {value: P}}}
5         self.domains = {} # {node: [possible_values]}
6
7     def add_variable(self, node, parents, domain, cpt):
8         """
9         Them bien voi CPT
10        - node: ten bien
11        - parents: danh sach cha
12        - domain: danh sach gia tri co the (list)
13        - cpt: dict {tuple(parent_values): {value: prob}}
14              Vi du: {"Weather": ["Sunny", "Rainy"],
15                    CPT = {(): {"Sunny": 0.7, "Rainy": 0.3}}}
16        """
17        self.add_node(node)
18        for p in parents:
19            self.add_edge(p, node)
20        self.cpt[node] = cpt
21        self.domains[node] = domain
22
23    def probability(self, node, value, assignment):
24        """Tra ve P(node=value | parents)"""
25        parents = self.get_parents(node)
26        parent_values = tuple(assignment[p] for p in parents)
27        return self.cpt[node][parent_values][value]
28
29    def joint_probability(self, assignment):
30        """Tinh P(assignment) = tích P(node | parents)"""
31        prob = 1.0
32        for node in self.adj:
33            prob *= self.probability(node, assignment[node],
34                                     assignment)
35        return prob
36
37    def query(self, Q, evidence):
38        """
39        Truy van P(Q | evidence)
```

```
39     - Q: bien can hoi
40     - evidence: dict {node: value}
41     """
42     dist = {v: 0.0 for v in self.domains[Q]}
43
44     # duyet tat ca to hop gia tri cho cac bien
45     for values in itertools.product(*[self.domains[v] for v in
46         self.adj]):
47         assignment = dict(zip(self.adj.keys(), values))
48         # check khop evidence
49         if all(assignment[e] == evidence[e] for e in evidence):
50             dist[assignment[Q]] += self.joint_probability(
51                 assignment)
52
53     # chuan hoa
54     norm = sum(dist.values())
55     for k in dist:
56         dist[k] /= norm
57     return dist
58
59 def approximate_query(self, Q, evidence, N=1000):
60     dist = {v: 0.0 for v in self.domains[Q]}
61     nodes = self.topological_sort()
62
63     for _ in range(N):
64         weight = 1.0
65         sample = {}
66
67         for node in nodes:
68             parents = self.get_parents(node)
69             parent_values = tuple(sample[p] for p in parents)
70
71             probs = self.cpt[node][parent_values]
72
73             if node in evidence:
74                 val = evidence[node]
75                 sample[node] = val
76                 weight *= probs[val] # nhan trong so
77             else:
78                 # chon gia tri ngau nhien theo phan phoi
79                 values, pvals = zip(*probs.items())
80                 val = random.choices(values, weights=pvals, k=1)
81                 sample[node] = val
```



```
81         dist[sample[Q]] += weight
82
83     # chuan hoa
84     norm = sum(dist.values())
85     for k in dist:
86         dist[k] /= norm
87     return dist
88
89 def log_query(self, Q, evidence):
90     import math
91     import itertools
92
93     def _logaddexp(a, b):
94         if a == float('-inf'): return b
95         if b == float('-inf'): return a
96         m = a if a > b else b
97         return m + math.log(math.exp(a - m) + math.exp(b - m))
98
99     def _logsumexp_list(values):
100         out = float('-inf')
101         for v in values:
102             out = _logaddexp(out, v)
103         return out
104
105     # init log distribution
106     log_dist = {v: float('-inf') for v in self.domains[Q]}
107     nodes = list(self.adj.keys())
108
109     # log CPT
110     def log_prob(node, val, assignment):
111         parents = self.get_parents(node)
112         parent_vals = tuple(assignment[p] for p in parents)
113         p = self.cpt[node][parent_vals][val]
114         return math.log(p)
115
116     # log joint
117     def log_joint(assignment):
118         s = 0.0
119         for node in nodes:
120             s += log_prob(node, assignment[node], assignment)
121         return s
122
123     # enumerate assignments
124     for values in itertools.product(*[self.domains[v] for v in
        nodes]):
```

```
125         assignment = dict(zip(nodes, values))
126
127         # evidence check
128         if any(assignment[e] != evidence[e] for e in evidence):
129             continue
130
131         q_val = assignment[Q]
132         lj = log_joint(assignment)
133
134         # accumulate with logaddexp
135         log_dist[q_val] = _logaddexp(log_dist[q_val], lj)
136
137         # normalization via logsumexp
138         log_Z = _logsumexp_list(log_dist.values())
139         for v in log_dist:
140             log_dist[v] -= log_Z
141
142     return log_dist
143
144
145
146 def visualize(self, hide_table = False):
147     """Ve mang Bayesian voi CPT cho tung node"""
148     G = nx.DiGraph()
149
150     # them cac node va canh
151     for node in self.adj:
152         G.add_node(node)
153         for child in self.adj[node]:
154             G.add_edge(node, child)
155
156     pos = nx.spring_layout(G, seed=42) # vi tri node
157     plt.figure(figsize=(12, 8))
158
159     # ve graph
160     nx.draw_networkx_nodes(G, pos, node_size=1800, node_color="
        lightblue")
161     nx.draw_networkx_labels(G, pos, font_size=10, font_weight="
        bold")
162
163     # ve canh voi mui ten ro rang
164     nx.draw_networkx_edges(G, pos,
165         arrowstyle="->",
166         arrowsize=20,
167         width=2,
```

```
168         min_source_margin=20,      # lui khoi tam node nguon
169         min_target_margin=20      # lui khoi tam node dich
170     )
171
172     if not hide_table:
173         # ve CPT gan node
174         for node, (x, y) in pos.items():
175             cpt_str = self._format_cpt(node)
176             plt.text(
177                 x, y - 0.15, cpt_str,
178                 ha="center", va="top",
179                 fontsize=8,
180                 bbox=dict(boxstyle="round", facecolor="wheat",
181                           alpha=0.5)
182             )
183
184     plt.axis("off")
185     plt.show()
186
187     def _format_cpt(self, node):
188         """Chuyen CPT cua node thanh string gon gang"""
189         lines = []
190         for parent_vals, probs in self.cpt[node].items():
191             parent_str = ", ".join(f"{p}={v}" for p, v in zip(self.
192                 get_parents(node), parent_vals))
193             if not parent_str:
194                 parent_str = "    "
195             prob_str = ", ".join(f"{val}:{p:.2f}" for val, p in probs.
196                 items())
197             lines.append(f"[{parent_str}] -> {prob_str}")
198         return "\n".join(lines)
```

3 Tổng quan về dữ liệu

3.1 Nguồn dữ liệu

Nguồn dữ liệu được sử dụng cho bài toán này là bộ **Student Depression Dataset**. Đây là bộ dữ liệu thu thập thông tin về tình trạng tâm lý của sinh viên, bao gồm các thuộc tính nhân khẩu học, học tập và các chỉ số sức khỏe tinh thần.



```
import os
df = pd.read_csv(os.path.join(path, "student_depression_dataset.csv"))
df.head()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts?	Work/Study Hours	Financial Stress	Family History of Mental Illness	Depression
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	'5-6 hours'	Healthy	B.Pharm	Yes	3.0	1.0	No	1
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	'5-6 hours'	Moderate	BSc	No	3.0	2.0	Yes	0
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	'Less than 5 hours'	Healthy	BA	No	9.0	1.0	Yes	0
3	30	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	'7-8 hours'	Moderate	BCA	Yes	4.0	5.0	Yes	1
4	32	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	'5-6 hours'	Moderate	M.Tech	Yes	1.0	1.0	No	0

Hình 1: Một số mẫu trong tập dữ liệu Student Depression Dataset

Một cái nhìn sơ bộ về dữ liệu cho thấy gồm 18 thuộc tính bao gồm:

- **id**: Mã số riêng biệt cho từng cá nhân (int64).
- **Gender**: Giới tính (object).
- **Age**: Tuổi của sinh viên (float64).
- **City**: Nơi sinh sống (object).
- **Profession**: Lĩnh vực học tập, làm việc, nghiên cứu (object).
- **Academic Pressure**: Mức độ áp lực học tập (float64).
- **Work Pressure**: Mức độ áp lực công việc (float64).
- **CGPA**: Điểm trung bình tích lũy (float64).
- **Study Satisfaction**: Mức độ hài lòng trong học tập (float64).
- **Job Satisfaction**: Mức độ hài lòng trong công việc (float64).
- **Sleep Duration**: Thời lượng ngủ trung bình mỗi ngày (object).
- **Dietary Habits**: Thói quen ăn uống (object).
- **Degree**: Trình độ học vấn hoặc chương trình học hiện tại (object).
- **Have you ever had suicidal thoughts?**: Đã từng có suy nghĩ tự tử hay chưa (object).
- **Work/Study Hours**: Số giờ làm việc hoặc học tập mỗi ngày (float64).
- **Financial Stress**: Mức độ căng thẳng tài chính (object).

- **Family History of Mental Illness:** Tiền sử gia đình có người mắc bệnh tâm lý hay không (object).
- **Depression:** Trạng thái trầm cảm (biến mục tiêu) (int64).

```
print("\nDataset Info:")
print(f"Total samples: {len(df):,}")
print(f"Features: {df.shape[1]}")
print(f"Data Shape: {df.shape}")
print(f"Columns: {df.columns}")

Dataset Info:
Total samples: 27,901
Features: 18
Data Shape: (27901, 18)
Columns: Index(['id', 'Gender', 'Age', 'City', 'Profession', 'Academic Pressure',
               'Work Pressure', 'CGPA', 'Study Satisfaction', 'Job Satisfaction',
               'Sleep Duration', 'Dietary Habits', 'Degree',
               'Have you ever had suicidal thoughts ?', 'Work/Study Hours',
               'Financial Stress', 'Family History of Mental Illness', 'Depression'],
              dtype='object')
```

Hình 2: Các thông số cơ bản của tập dữ liệu Student Depression Dataset

```
print("Dtypes:")
print(df.dtypes)

Dtypes:
id                int64
Gender            object
Age              float64
City             object
Profession        object
Academic Pressure float64
Work Pressure     float64
CGPA             float64
Study Satisfaction float64
Job Satisfaction  float64
Sleep Duration    object
Dietary Habits    object
Degree           object
Have you ever had suicidal thoughts ? object
Work/Study Hours  float64
Financial Stress  object
Family History of Mental Illness object
Depression        int64
dtype: object
```

Hình 3: Kiểu dữ liệu của các đặc trưng trong Student Depression Dataset

4 Tiền xử lý số liệu

4.1 Kiểm tra Missing value

Đầu tiên chúng tôi thực hiện việc kiểm tra các dữ liệu bị thiếu (missing values). Kết quả cho thấy bộ dữ liệu rất sạch và không có bất kỳ giá trị bị thiếu nào trong tất cả cột. Điều này giúp đơn giản hóa quá trình xử lý, không cần thực hiện các kỹ thuật điền khuyết.

```
def missing_report(df: pd.DataFrame) -> pd.DataFrame:
    missing_df = df.isna().sum().sort_values(ascending=False)
    return pd.DataFrame({"missing": missing_df, "percent": (missing_df / len(df) * 100).round(2)})

print("Missing Value:\n", missing_report(df))
```

Missing Value:

	missing	percent
id	0	0.0
Gender	0	0.0
Age	0	0.0
City	0	0.0
Profession	0	0.0
Academic Pressure	0	0.0
Work Pressure	0	0.0
CGPA	0	0.0
Study Satisfaction	0	0.0
Job Satisfaction	0	0.0
Sleep Duration	0	0.0
Dietary Habits	0	0.0
Degree	0	0.0
Have you ever had suicidal thoughts ?	0	0.0
Work/Study Hours	0	0.0
Financial Stress	0	0.0
Family History of Mental Illness	0	0.0
Depression	0	0.0

Hình 4: Kiểm tra sự tồn tại missing value

4.2 Kiểm tra số lượng giá trị khác nhau

Kế tiếp chúng tôi thực hiện việc kiểm tra số lượng các giá trị khác nhau trong bộ dữ liệu.

- **id (27901)**: Mỗi cá nhân có một mã định danh riêng biệt, không bị trùng lặp.
- **Age (34)**: Có 34 giá trị tuổi khác nhau, phân bố trên nhiều nhóm tuổi.
- **Gender (2), City (52), Profession (14), Sleep Duration (5), Dietary Habits (4), Degree (28)**: Các biến phân loại (*categorical*) có số lượng giá trị khác nhau, cần được mã hoá (Label Encoding hoặc One-Hot Encoding) trước khi đưa vào mô hình.
- **Academic Pressure (6), Work Pressure (3), CGPA (332), Study Satisfaction (6), Job Satisfaction (5), Work/Study Hours (13)**: Các biến liên tục hoặc thứ bậc (*numeric/ordinal*), có độ phân tán phù hợp để đưa vào mô hình phân tích.
- **Have you ever had suicidal thoughts? (2), Financial Stress (6), Family History of Mental Illness (2), Depression (2)**: Các biến nhị phân (0/1 hoặc Yes/No), thể hiện tình trạng có hoặc không, có thể được giữ nguyên dưới dạng số khi huấn luyện mô hình Mạng Bayes.

```
[ ] print(df.nunique())
✓ Os
id                27901
Gender              2
Age                34
City               52
Profession         14
Academic Pressure   6
Work Pressure       3
CGPA               332
Study Satisfaction  6
Job Satisfaction    5
Sleep Duration      5
Dietary Habits      4
Degree            28
Have you ever had suicidal thoughts ? 2
Work/Study Hours    13
Financial Stress     6
Family History of Mental Illness      2
Depression          2
dtype: int64
```

Hình 5: Kiểm tra số lượng giá trị khác nhau trong mỗi đặc trưng

5 Thống kê tả đối với các biến numeric

5.1 Đánh giá tổng quan

Dựa trên các chỉ số thống kê cơ bản và biểu đồ Histogram của các thuộc tính dạng số trong bộ dữ liệu, ta có một số nhận xét như sau:

- **Độ tuổi (Age)** của các cá nhân trong bộ dữ liệu dao động từ **18 đến 59 tuổi**, với giá trị trung bình khoảng **25.8 tuổi** và độ lệch chuẩn là **4.9**. Điều này cho thấy phần lớn mẫu dữ liệu tập trung ở nhóm tuổi sinh viên và người trẻ.
- **Áp lực học tập (Academic Pressure)** có giá trị trung bình là **3.14** trên thang điểm 0–5, chứng tỏ đa số sinh viên cảm nhận mức áp lực học tập trung bình đến cao.
- **Áp lực công việc (Work Pressure)** có trung bình rất nhỏ (**0.00043**) và độ lệch chuẩn thấp, cho thấy phần lớn sinh viên không chịu áp lực đáng kể từ công việc, có thể do phần lớn chưa đi làm hoặc công việc không phải là yếu tố chính.
- **Điểm trung bình tích lũy (CGPA)** dao động từ **0.0 đến 10.0**, với trung bình **7.65** và độ lệch chuẩn **1.47**, phản ánh mức học lực tương đối khá của phần lớn sinh viên.
- **Mức độ hài lòng trong học tập (Study Satisfaction)** có trung bình khoảng **2.94** trên thang 0–5, cho thấy phần lớn sinh viên cảm thấy hài lòng ở mức trung bình.
- **Mức độ hài lòng trong công việc (Job Satisfaction)** có giá trị trung bình gần bằng 0 (**0.00068**), cho thấy yếu tố này ít xuất hiện hoặc chỉ có ý nghĩa với một nhóm nhỏ trong tập dữ liệu.

- Số giờ học/làm việc mỗi ngày (Work/Study Hours) có giá trị trung bình khoảng **7.16 giờ/ngày**, với độ lệch chuẩn **3.71**, phản ánh sự đa dạng trong thời gian học tập và làm việc của sinh viên.

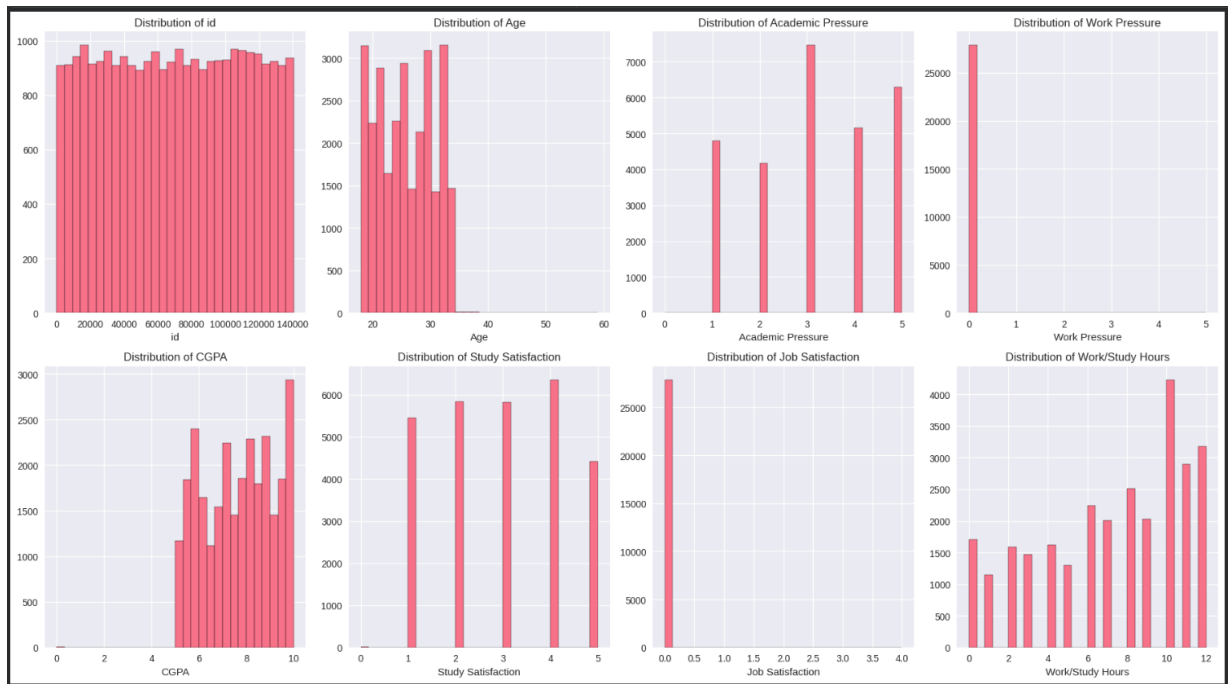
Nhìn chung, các thống kê mô tả cho thấy dữ liệu có phân bố hợp lý, phản ánh thực tế đời sống học tập và tâm lý của sinh viên. Những thuộc tính như **Academic Pressure**, **Study Satisfaction** và **CGPA** nhiều khả năng có vai trò quan trọng trong việc dự đoán tình trạng **Depression**.

```
[ ] numeric_feature = [  
    "id", "Age", "Academic Pressure", "Work Pressure", "CGPA",  
    "Study Satisfaction", "Job Satisfaction",  
    "Work/Study Hours"  
]  
  
print(df[numeric_feature].describe(include=[np.float64, np.int64]))
```

	id	Age	Academic Pressure	Work Pressure \
count	27901.000000	27901.000000	27901.000000	27901.000000
mean	70442.149421	25.822300	3.141214	0.000430
std	40641.175216	4.905687	1.381465	0.043992
min	2.000000	18.000000	0.000000	0.000000
25%	35039.000000	21.000000	2.000000	0.000000
50%	70684.000000	25.000000	3.000000	0.000000
75%	105818.000000	30.000000	4.000000	0.000000
max	140699.000000	59.000000	5.000000	5.000000

	CGPA	Study Satisfaction	Job Satisfaction	Work/Study Hours
count	27901.000000	27901.000000	27901.000000	27901.000000
mean	7.656104	2.943837	0.000681	7.156984
std	1.470707	1.361148	0.044394	3.707642
min	0.000000	0.000000	0.000000	0.000000
25%	6.290000	2.000000	0.000000	4.000000
50%	7.770000	3.000000	0.000000	8.000000
75%	8.920000	4.000000	0.000000	10.000000
max	10.000000	5.000000	4.000000	12.000000

Hình 6: Tóm tắt các thông tin về bộ dữ liệu



Hình 7: Các biểu đồ Histogram

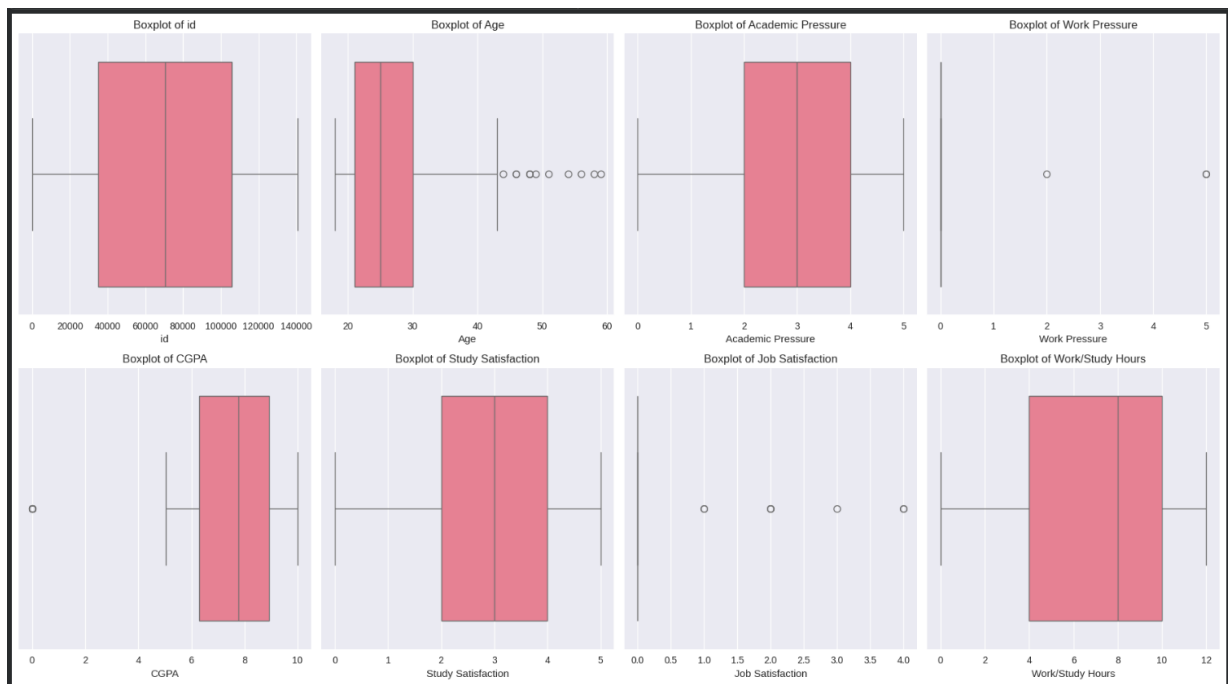
5.2 Phân tích biểu đồ boxplot

Phân tích biểu đồ **Boxplot** và kiểm tra số lượng Outliers cho thấy xu hướng lệch và tồn tại các giá trị ngoại lai của các biến số trong bộ dữ liệu. Kết quả thu được cho thấy:

- **id**: Phân phối khá đồng đều và đối xứng, không xuất hiện giá trị ngoại lai đáng kể. Biến này chỉ mang tính định danh, không có ý nghĩa trong phân tích mô hình.
- **Age**: Phân bố chủ yếu tập trung trong khoảng **20–30 tuổi**, phù hợp với đặc điểm của nhóm sinh viên. Một vài giá trị ngoại lai xuất hiện ở độ tuổi cao hơn, tuy nhiên số lượng không đáng kể so với phần còn lại.
- **Academic Pressure**: Giá trị chủ yếu nằm trong khoảng **2–4**, phân bố có xu hướng **lệch phải**, cho thấy phần lớn sinh viên cảm nhận áp lực học tập ở mức trung bình đến cao.
- **Work Pressure**: Phần lớn giá trị tập trung ở mức **0**, chỉ có một vài trường hợp cá biệt với giá trị cao hơn. Hiện tượng này có thể lý giải do đối tượng khảo sát chủ yếu là sinh viên, chưa thực sự chịu áp lực từ công việc.
- **CGPA**: Các giá trị phân bố chủ yếu quanh mức **8.0**, với xu hướng **lệch phải**, phản ánh rằng phần lớn sinh viên đạt điểm trung bình tích lũy khá cao.

- **Study Satisfaction:** Tập trung chủ yếu ở mức **2–4** và có xu hướng **lệch phải**, cho thấy đa phần sinh viên cảm thấy hài lòng ở mức trung bình trở lên đối với quá trình học tập.
- **Job Satisfaction:** Phần lớn dữ liệu nằm ở mức **0**, chỉ xuất hiện một số ít giá trị cao hơn. Tương tự như biến **Work Pressure**, điều này phản ánh đặc điểm của nhóm khảo sát là sinh viên, chưa tham gia nhiều vào thị trường lao động.
- **Work/Study Hours:** Thời gian học tập hoặc làm việc chủ yếu dao động trong khoảng **4–10 giờ mỗi ngày**, với phân bố **lệch phải**, cho thấy sự đa dạng trong cường độ học tập và làm việc giữa các cá nhân.

Tổng thể, kết quả Boxplot cho thấy phần lớn các biến số có phân bố hợp lý, không xuất hiện nhiều giá trị ngoại lai nghiêm trọng. Một số biến như **Academic Pressure**, **Study Satisfaction** và **Work/Study Hours** thể hiện sự khác biệt rõ rệt trong mức độ phân bố, gợi ý rằng chúng có thể đóng vai trò quan trọng trong việc phân tích và dự đoán tình trạng **Depression**.



Hình 8: Các biểu đồ Boxplot

```
{ }
0s ▶ outlier_counts = {}

for col in numeric_feature:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
    outlier_counts[col] = len(outliers)

# Hiển thị kết quả
for col, count in outlier_counts.items():
    print(f"{col}: {count} outliers")

id: 0 outliers
Age: 12 outliers
Academic Pressure: 0 outliers
Work Pressure: 3 outliers
CGPA: 9 outliers
Study Satisfaction: 0 outliers
Job Satisfaction: 8 outliers
Work/Study Hours: 0 outliers
```

Hình 9: Kiểm tra Outlier

6 Thống kê tả đối với các biến phân loại

Dựa trên kết quả thống kê tần suất của các biến phân loại trong bộ dữ liệu, ta có một số nhận xét sau:

- **Gender:** Tập dữ liệu có sự phân bố giới tính tương đối cân đối với **15,547 nam** và **12,354 nữ**. Điều này giúp mô hình tránh được thiên lệch giới tính khi huấn luyện.
- **City:** Có tổng cộng **52 thành phố** khác nhau được ghi nhận. Các thành phố có số lượng người tham gia cao nhất gồm *Kalyan* (1,570), *Srinagar* (1,372), *Hyderabad* (1,340) và *Vasai-Virar* (1,290). Phân bố đa dạng này phản ánh tính đại diện tốt về mặt địa lý của tập dữ liệu.
- **Profession:** Biến nghề nghiệp có **14 giá trị khác nhau**, trong đó **Student** chiếm gần như toàn bộ dữ liệu (**27,870 mẫu**). Các nghề khác như *Architect*, *Teacher*, *Doctor*, *Pharmacist* chỉ chiếm tỷ lệ rất nhỏ. Điều này cho thấy bộ dữ liệu tập trung chủ yếu vào nhóm đối tượng học sinh, sinh viên — phù hợp với mục tiêu nghiên cứu về trầm cảm trong môi trường học tập.
- **Sleep Duration:** Có **5 nhóm thời lượng ngủ** khác nhau. Phần lớn người tham gia ngủ dưới **5 giờ (8,310)** hoặc **7–8 giờ (7,346)**, tiếp theo là nhóm **5–6 giờ (6,183)** và hơn **8 giờ (6,044)**. Điều này cho thấy đa số sinh viên có thời lượng ngủ ngắn hoặc vừa phải, có thể là yếu tố liên quan đến sức khỏe tinh thần.
- **Dietary Habits:** Thói quen ăn uống được chia thành **4 nhóm**, trong đó **Unhealthy** chiếm tỷ lệ cao nhất (**10,317 mẫu**), tiếp đến là **Moderate (9,921)** và **Healthy (7,651)**. Xu hướng này cho thấy phần lớn sinh viên chưa có chế độ ăn lành mạnh, có thể ảnh hưởng đến mức độ căng thẳng và trầm cảm.

- **Degree:** Có 28 chương trình đào tạo khác nhau, nổi bật nhất là **Class 12** (6,080), **B.Ed** (1,867), **B.Com** (1,506), **B.Arch** (1,478), **BCA** (1,433), và **MSc** (1,190). Điều này cho thấy dữ liệu bao phủ cả bậc trung học và đại học, phản ánh đa dạng trình độ học vấn.
- **Have you ever had suicidal thoughts?:** Khoảng **17,656 người (63.3%)** thừa nhận từng có suy nghĩ tự tử, trong khi **10,245 người (36.7%)** trả lời không. Đây là một tỷ lệ đáng chú ý, thể hiện tầm quan trọng của việc phân tích yếu tố này trong mối liên hệ với trầm cảm.
- **Financial Stress:** Có 6 mức độ căng thẳng tài chính, với giá trị phổ biến nhất là **5.0 (6,715 mẫu)**, tiếp theo là **4.0 (5,775)** và **3.0 (5,226)**. Điều này cho thấy phần lớn sinh viên đang chịu **mức độ căng thẳng tài chính trung bình đến cao**. Ngoài ra còn có 3 giá trị khi không xác định được mức Financial Stress “?”.

Nhìn chung, các biến phân loại thể hiện sự đa dạng và tính đại diện tốt về nhân khẩu học, học vấn và lối sống. Tuy nhiên, sự tập trung cao ở nhóm **Student** và các giá trị lệch như **Work Pressure = 0** hay **Job Satisfaction = 0** cho thấy dữ liệu có thể mang thiên hướng mạnh về nhóm sinh viên, điều này cần được cân nhắc trong quá trình xây dựng mô hình Mạng Bayes để đảm bảo kết quả suy luận chính xác.

7 Lựa chọn các cột cho mô hình

Sau khi khảo sát sơ bộ, nhóm tiến hành bước tiền xử lý dữ liệu nhằm đảm bảo chất lượng và tính phù hợp cho mô hình dự đoán. Để dễ dàng hơn trong việc loại bỏ các biến, nhóm đã sử dụng biểu đồ nhiệt, Mutual Information - MI và kiến thức sẵn có để làm cơ sở lựa chọn các biến. Các biến được xem xét kỹ lưỡng dựa trên mức độ liên quan đến hiện tượng trầm cảm.

Các biến được giữ lại

Những đặc trưng có khả năng là nguyên nhân trực tiếp hoặc gián tiếp dẫn đến trầm cảm được giữ lại, bao gồm:

- **Work/Study Hours:** Thời gian học tập hoặc làm việc.
- **Financial Stress:** Mức độ căng thẳng tài chính.
- **Academic Pressure:** Áp lực học tập.
- **CGPA:** Điểm trung bình tích lũy.
- **Study Satisfaction:** Mức độ hài lòng trong học tập.



- **Have you ever had suicidal thoughts?:** Biến thể hiện tình trạng từng có ý định tự tử.
- **Depression:** Nhân mục tiêu — có hoặc không bị trầm cảm.

Các biến bị loại bỏ

Các biến sau được loại bỏ do không có ý nghĩa thống kê, không liên quan trực tiếp đến vấn đề nghiên cứu, hoặc có dữ liệu không hợp lý:

- **id:** Chỉ mang ý nghĩa định danh, không liên quan đến trầm cảm.
- **Age:** Tuổi không có ảnh hưởng đáng kể đến trầm cảm trong nhóm mẫu.
- **Work Pressure:** Đa số đối tượng là học sinh, sinh viên nên biến này không phản ánh chính xác áp lực công việc.
- **Job Satisfaction:** Tương tự như trên, biến này thiếu ý nghĩa với nhóm chưa đi làm.
- **Gender:** Giới tính không thể hiện mối liên hệ rõ ràng với trầm cảm trong bộ dữ liệu này.
- **City:** Chỉ phản ánh nơi sinh sống, không mang tính nhân quả trong hiện tượng trầm cảm.
- **Profession:** Phần lớn mẫu là sinh viên, nên biến này không mang thông tin phân biệt có ý nghĩa.
- **Sleep Duration:** Được xem là *kết quả* của trầm cảm hơn là nguyên nhân.
- **Dietary Habits:** Không có mối quan hệ rõ ràng với các yếu tố học tập hoặc tâm lý.
- **Degree:** Không ảnh hưởng trực tiếp hoặc gián tiếp đến trạng thái trầm cảm.
- **Family History of Mental Illness:** Không thể hiện ảnh hưởng đáng kể trong dữ liệu thu thập được.

Như vậy, sau quá trình sàng lọc, chỉ giữ lại các biến có tính giải thích và giá trị thực tiễn cao phục vụ cho quá trình xây dựng mô hình dự đoán trầm cảm.



Hình 10: Biểu đồ Heatmap

```
[ ] ▶ from sklearn.feature_selection import mutual_info_classif
import pandas as pd

categorical_feature = [
    "Gender", "City", "Profession", "Sleep Duration", "Dietary Habits",
    "Degree", "Have you ever had suicidal thoughts ?",
    "Financial Stress", "Family History of Mental Illness"
]

X_cat = df[categorical_feature].copy()
y = df["Depression"]

for col in X_cat.columns:
    X_cat[col] = pd.factorize(X_cat[col])[0]

mi = mutual_info_classif(X_cat, y, discrete_features=True, random_state=42)

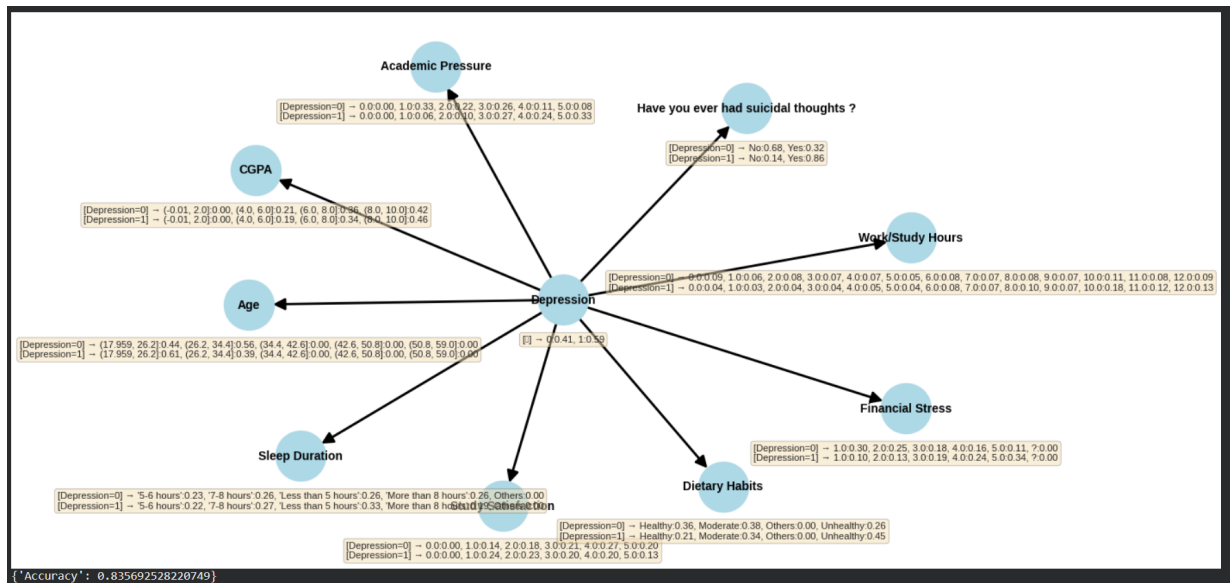
mi_series = pd.Series(mi, index=categorical_feature).sort_values(ascending=False)
print(mi_series)
```

Have you ever had suicidal thoughts ?	0.154642
Financial Stress	0.068728
Dietary Habits	0.021837
Degree	0.009772
Sleep Duration	0.004962
City	0.003534
Family History of Mental Illness	0.001429
Profession	0.000359
Gender	0.000002
dtype: float64	

Hình 11: Mutual Information - MI

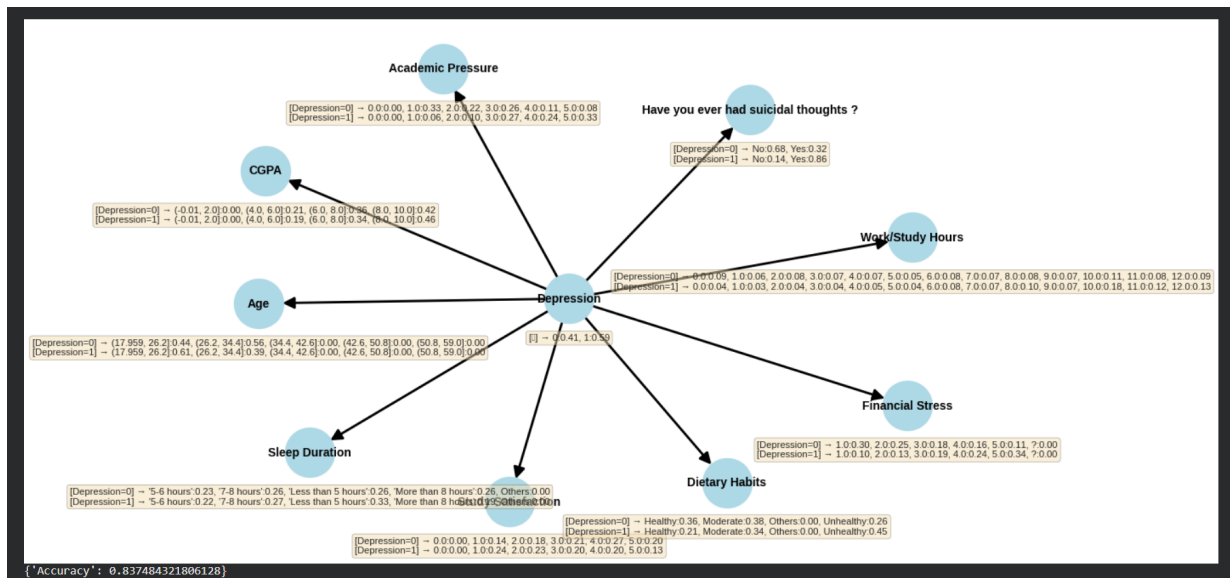
8 Huấn luyện mô hình

Ở bước này chúng ta sẽ hiện thực mô các mô hình mạng Bayesian khác nhau:



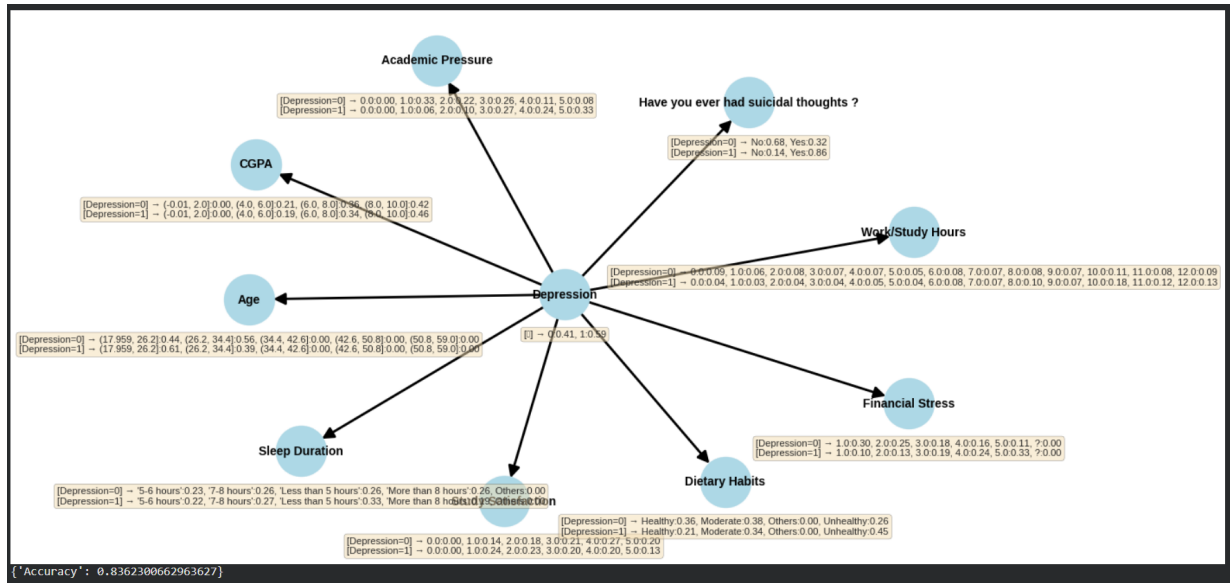
Hình 12: Mạng Bayesian tính toán xấp xỉ dùng MLE với smoothing = 0

Hình trên minh họa mạng Bayesian được huấn luyện bằng phương pháp **Maximum Likelihood Estimation (MLE)** mà không sử dụng hệ số làm trơn (*smoothing* = 0). Kết quả Accuracy = 0.8331840172012184.



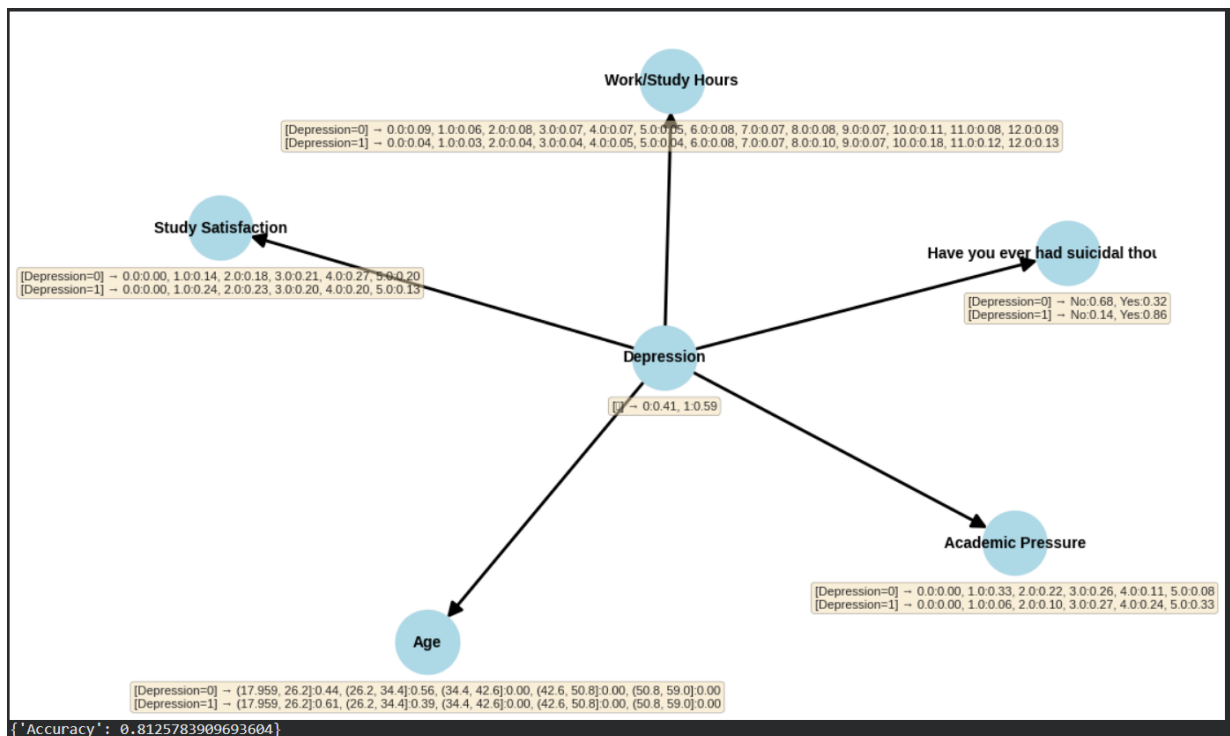
Hình 13: Mạng Bayesian tính toán xấp xỉ dùng MLE với smoothing = 0.1

Hình trên thể hiện mạng Bayesian khi huấn luyện bằng MLE với hệ số làm trơn *smoothing* = 0.1. Kết quả Accuracy = 0.8362300662963627.



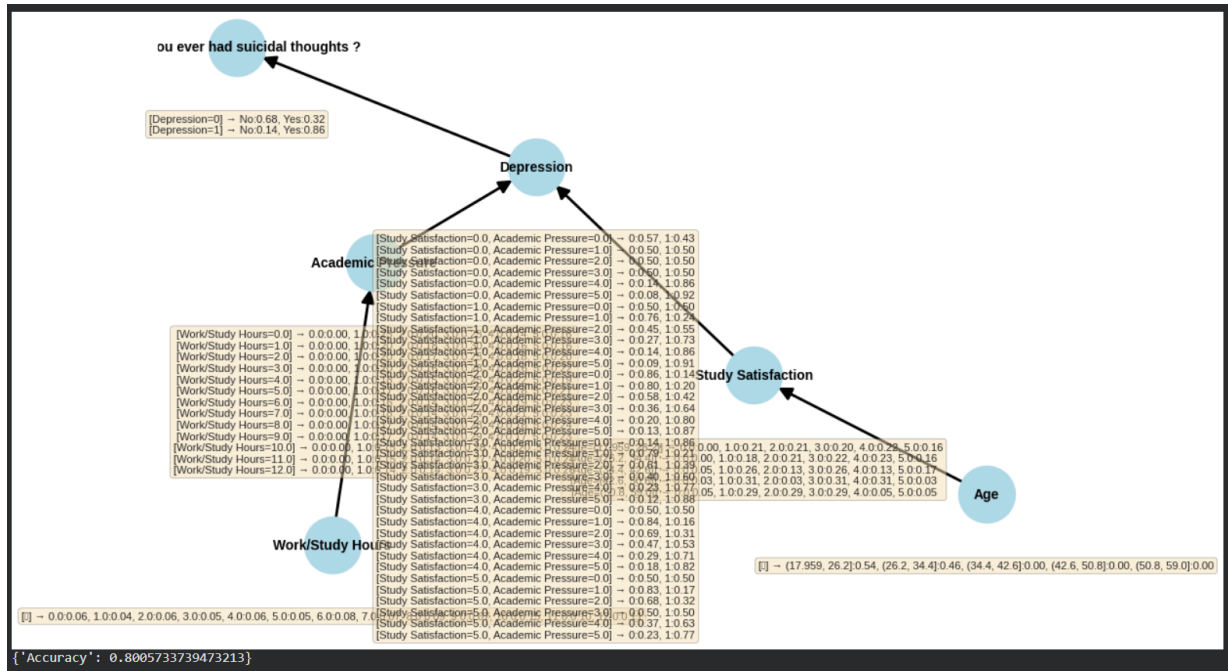
Hình 14: Mạng Bayesian tính toán xấp xỉ dùng MAP

Hình trên biểu diễn kết quả huấn luyện mạng Bayesian theo phương pháp **Maximum A Posteriori (MAP)**. Kết quả Accuracy = 0.8364092456549006.



Hình 15: Mạng Bayesian tính toán chính xác dùng MLE với smoothing = 0.2

Hình trên minh họa mạng khi **chính xác (exact inference)** trong mạng Bayesian, sử dụng phương pháp MLE với hệ số làm tròn *smoothing* = 0.2. Kết quả Accuracy = 0.8125783909693604.



Hình 16: Mạng Bayesian tính toán Log-Likelihood dùng MLE với *smoothing* = 0.2

Hình trên biểu diễn quá trình đánh giá mô hình thông qua **hàm log-likelihood**, sử dụng phương pháp MLE với hệ số làm tròn *smoothing* = 0.2. Kết quả Accuracy = 0.8005733739473213.

9 Kết luận

Từ việc thực hành phần mở rộng này, nhóm đã có thêm nhiều kiến thức và kinh nghiệm thực tiễn trong việc ứng dụng **mạng Bayesian (Bayesian Network)** vào một bài toán cơ bản.

Việc triển khai các phương pháp như **Maximum Likelihood Estimation (MLE)** và **Maximum A Posteriori (MAP)** giúp nhóm nắm bắt được sự khác biệt giữa hai hướng tiếp cận: trong khi MLE chỉ dựa trên dữ liệu quan sát để ước lượng tham số, thì MAP kết hợp thêm thông tin tiên nghiệm (*prior*) nhằm tăng độ tin cậy khi dữ liệu bị thiếu hoặc nhiễu. Ngoài ra, nhóm cũng nhận thấy rằng việc áp dụng **smoothing** đóng vai trò rất quan trọng trong thực tế – giúp tránh hiện tượng xác suất bằng 0, cải thiện khả năng khái quát và tính ổn định của mô hình.

Bên cạnh đó, thông qua việc so sánh giữa *xấp xỉ suy luận* (*approximate inference*) và *suy luận chính xác* (*exact inference*), nhóm đã nhận ra sự đánh đổi giữa **độ chính xác** và **hiệu năng tính toán**. Các phương pháp xấp xỉ có thể cho kết quả nhanh và đủ tốt trong các hệ thống lớn, trong khi suy luận chính xác lại cần nhiều thời gian và tài nguyên hơn nhưng đảm bảo độ tin cậy cao.

Cuối cùng, việc phân tích và trực quan hóa các kết quả thông qua các biểu đồ log-likelihood cũng giúp nhóm hiểu sâu hơn về ý nghĩa của các chỉ số đánh giá mô hình xác suất. Toàn bộ quá trình này không chỉ giúp củng cố kiến thức lý thuyết về mạng Bayesian, mà còn rèn luyện kỹ năng triển khai mô hình thống kê trong môi trường thực hành, đặt nền tảng cho việc ứng dụng các kỹ thuật học máy xác suất trong các bài toán phức tạp hơn sau này.

10 Tài liệu tham khảo

Tài liệu

- [1] T. Koski and J. Noble, *Bayesian Networks: An Introduction*, Chichester, UK: John Wiley & Sons, 2009.